

# CLOUDFRONT FOR FRONTEND & APIs

---

Single Domain • Dual Origin • WAF • TLS • Security

Complete Production Setup with CloudFront, S3, ALB & ECS

CloudFront • WAF • S3 • ALB • ECS • ACM • OAC • VPC Origins

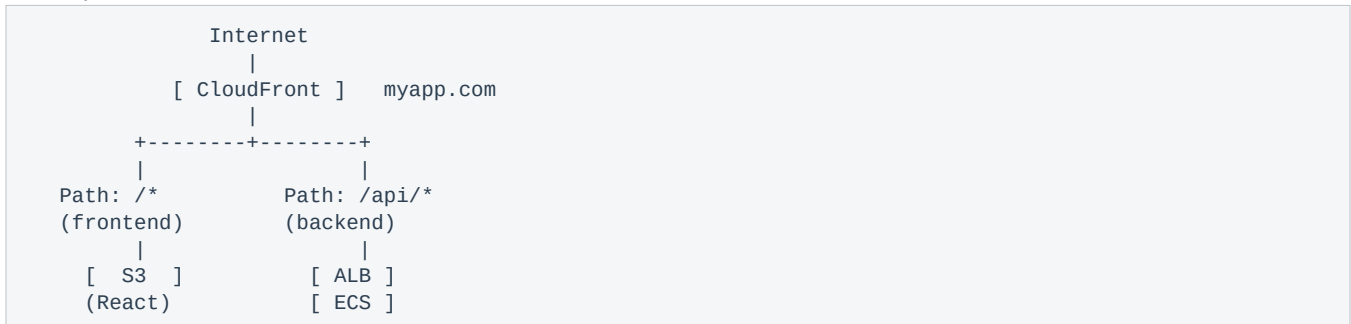
*Interview & System Design Preparation*

# TABLE OF CONTENTS

1. Architecture Overview
2. Key Concept: Origins and Behaviors
3. Setup: S3 for Frontend
4. Setup: ALB for Backend API
5. Setup: CloudFront Distribution
6. Cache Strategy: Frontend vs API
7. SPA Routing (React/Angular)
8. S3 Security: Origin Access Control (OAC)
9. CORS: Why Single Domain Solves It
10. Where TLS Termination Happens
11. WAF: Where It Fits
12. WAF: How to Configure
13. ALB Security: Restricting to CloudFront Only
14. CloudFront VPC Origins (Best Option)
15. Complete Security Stack
16. Complete Request Flow
17. Interview-Ready Answers

# 1. Architecture Overview

A single CloudFront distribution serves both the React/Angular frontend from S3 and the backend API from ALB. Users access one domain (myapp.com) and CloudFront routes requests to the correct origin based on the URL path.



This means myapp.com/ serves the React app, myapp.com/dashboard serves the React app (SPA routing), and myapp.com/api/orders hits the backend through ALB. One domain, no CORS issues, CDN-cached frontend, and dynamic API — all through one distribution.

## 2. Key Concept: Origins and Behaviors

CloudFront uses two concepts to route requests. An Origin is WHERE to fetch content from. A Behavior is WHICH requests go to WHICH origin, based on URL path patterns.

| Concept  | What It Is                | Example                 |
|----------|---------------------------|-------------------------|
| Origin   | Backend source of content | S3 bucket, ALB endpoint |
| Behavior | Path-based routing rule   | /api/* → ALB, /* → S3   |

**Important:** Behaviors are evaluated top to bottom. Specific paths (like /api/\*) must be listed BEFORE the default catch-all (\*). The default behavior always matches last.

| Priority    | Path Pattern        | Origin        | Cache                  |
|-------------|---------------------|---------------|------------------------|
| 1 (first)   | /api/*              | ALB (backend) | Disabled (dynamic)     |
| 2 (default) | * (everything else) | S3 (frontend) | Enabled (static files) |

### 3. Setup: S3 for Frontend

The React or Angular production build is deployed to an S3 bucket. The bucket is NOT configured as a static website and public access is completely blocked. Only CloudFront can read from it using Origin Access Control (OAC).

```
S3 Bucket: myapp-frontend
Static website hosting: DISABLED
Public access: BLOCKED
Contents:
  /index.html
  /static/js/main.a1b2c3.js   (hashed filename)
  /static/css/main.d4e5f6.css
  /favicon.ico
```

**Why block public access?** If S3 is public, anyone can access your files directly, bypassing CloudFront caching and WAF protection. OAC ensures only CloudFront reads from S3.

### 4. Setup: ALB for Backend API

The backend API runs on ECS behind an ALB. CloudFront treats the ALB as a Custom Origin and connects to it over HTTPS. The ALB is traditionally internet-facing because CloudFront connects over the public internet, but it should be locked down to accept only CloudFront traffic.

```
ALB: myapp-api-alb
Scheme: internet-facing
Listener: HTTPS 443 (ACM certificate)
Target Group: ECS tasks on port 8080 (HTTP)
```

## 5. Setup: CloudFront Distribution

### Origin 1 — S3 (Frontend)

- Origin Domain: myapp-frontend.s3.us-east-1.amazonaws.com
- Origin Access: OAC (Origin Access Control) — signs requests to S3
- Origin ID: s3-frontend

### Origin 2 — ALB (Backend API)

- Origin Domain: myapp-api-alb-123456.elb.amazonaws.com
- Protocol: HTTPS Only — encrypts traffic to ALB
- Custom Header: X-CloudFront-Secret = [random-value] — for ALB verification
- Origin ID: alb-backend

### Behavior 1 — API Traffic (/api/\*)

- Path Pattern: /api/\*
- Origin: alb-backend
- Allowed Methods: GET, HEAD, OPTIONS, PUT, POST, PATCH, DELETE
- Cache Policy: CachingDisabled — every request reaches backend
- Origin Request Policy: AllViewerExceptHostHeader — forwards all headers except Host

### Behavior 2 — Frontend (Default \*)

- Path Pattern: Default (\*)
- Origin: s3-frontend
- Allowed Methods: GET, HEAD
- Cache Policy: CachingOptimized — static files cached at edge
- Default Root Object: index.html

## 6. Cache Strategy: Frontend vs API

Frontend static files should be cached aggressively because they rarely change and have hashed filenames (main.a1b2c3.js). API responses should never be cached because they return dynamic data that changes with every request.

| Content          | Cache TTL              | Reason                                  |
|------------------|------------------------|-----------------------------------------|
| index.html       | 60 seconds or no-cache | Must pick up new deploys quickly        |
| static/js/*.js   | 1 year (31536000s)     | Hashed filename — new deploy = new file |
| static/css/*.css | 1 year                 | Same — hash changes on deploy           |
| images/*         | 1 year                 | Rarely change                           |
| /api/*           | CachingDisabled        | Dynamic data — must always be fresh     |

### Cache Invalidation After Deploy

After deploying a new frontend build, invalidate only index.html. JS and CSS files have content hashes in their filenames, so a new deploy produces new filenames automatically. Old files expire naturally from the cache.

```
aws cloudfront create-invalidation \  
  --distribution-id E1ABCDEF \  
  --paths '/index.html'
```

## 7. SPA Routing (React / Angular)

Single Page Applications handle routing on the client side. When a user visits `myapp.com/dashboard`, CloudFront looks for a `/dashboard` file in S3, which does not exist. S3 returns 403/404. The fix is to configure CloudFront Custom Error Responses to serve `index.html` for all missing paths, letting the React/Angular router handle the URL.

| Error Code            | Response Page | Response Code | TTL       |
|-----------------------|---------------|---------------|-----------|
| 403 (S3 missing file) | /index.html   | 200 OK        | 0 seconds |
| 404 (not found)       | /index.html   | 200 OK        | 0 seconds |

### The flow:

- User requests `/dashboard`
- CloudFront matches default behavior (\*) → S3 origin
- S3 has no `/dashboard` file → returns 403
- CloudFront error response → serves `/index.html` with 200
- Browser receives `index.html` → React Router sees `/dashboard` → renders Dashboard component



## 8. S3 Security: Origin Access Control (OAC)

OAC ensures only CloudFront can read from S3. CloudFront signs every request to S3. The S3 bucket policy allows only requests signed by your specific CloudFront distribution. Direct S3 URL access is blocked.

```
S3 Bucket Policy:
{
  "Effect": "Allow",
  "Principal": {
    "Service": "cloudfront.amazonaws.com"
  },
  "Action": "s3:GetObject",
  "Resource": "arn:aws:s3:::myapp-frontend/*",
  "Condition": {
    "StringEquals": {
      "AWS:SourceArn": "arn:aws:cloudfront::123456:distribution/E1ABC"
    }
  }
}
```

| Access Method                                             | Result    |
|-----------------------------------------------------------|-----------|
| Direct S3 URL: s3.amazonaws.com/myapp-frontend/index.html | BLOCKED ■ |
| CloudFront: myapp.com/index.html                          | ALLOWED ■ |

## 9. CORS: Why Single Domain Solves It

CORS issues occur when frontend and backend are on different domains (e.g., app.myapp.com calls api.myapp.com). By serving both through one CloudFront distribution on one domain, the frontend at myapp.com/ and the API at myapp.com/api/ share the same origin. No CORS configuration needed.

| Setup             | Frontend      | API            | CORS Needed?             |
|-------------------|---------------|----------------|--------------------------|
| Separate domains  | app.myapp.com | api.myapp.com  | YES ■ (different origin) |
| Single CloudFront | myapp.com/    | myapp.com/api/ | NO ■ (same origin)       |

## 10. Where TLS Termination Happens

TLS can terminate at multiple layers. The standard approach terminates at CloudFront for users and re-encrypts to ALB. TLS on ALB is important because CloudFront injects a custom header for verification — without HTTPS between CloudFront and ALB, this header travels unencrypted over the public internet and can be sniffed by attackers.

| Option     | Path                              | Security      | Use Case             |
|------------|-----------------------------------|---------------|----------------------|
| CF only    | User→HTTPS→CF→HTTP→ALB→HTTP→ECS   | Basic         | Non-sensitive apps   |
| CF + ALB   | User→HTTPS→CF→HTTPS→ALB→HTTP→ECS  | Recommended ■ | Most production apps |
| End-to-end | User→HTTPS→CF→HTTPS→ALB→HTTPS→ECS | Highest       | PCI-DSS, HIPAA       |

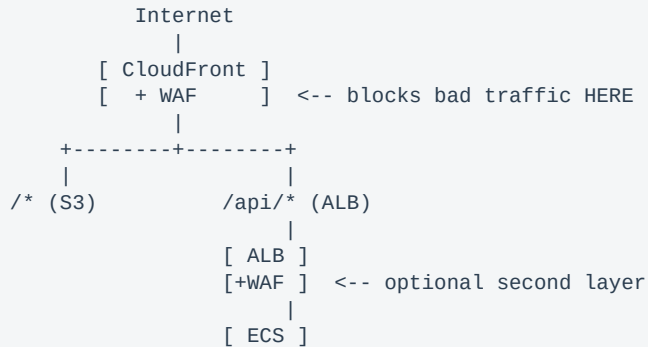
**Why TLS on ALB matters:** CloudFront injects a secret header (X-CloudFront-Secret) so ALB can verify requests came from CloudFront. Without HTTPS on ALB, this header travels in plain text over the internet. An attacker could sniff it and use it to bypass CloudFront. HTTPS encrypts the header in transit.

### CloudFront Origin Protocol Setting

```
CloudFront Origin:
  Protocol Policy: HTTPS Only
  HTTPS Port: 443
  Minimum SSL Protocol: TLSv1.2
```

## 11. WAF: Where It Fits

AWS WAF attaches to CloudFront as the primary protection layer. It blocks malicious traffic at the edge — before it reaches your ALB or ECS. An attacker in Tokyo is blocked at the Tokyo edge location. Bad traffic never enters your infrastructure.



| WAF Location         | Purpose                                | When to Use          |
|----------------------|----------------------------------------|----------------------|
| CloudFront (primary) | Block at edge, before infra            | Always — recommended |
| ALB (secondary)      | Defense in depth, API-specific rules   | High security apps   |
| Both                 | General rules on CF + API rules on ALB | Maximum protection   |

**Note:** WAF for CloudFront must be created in the us-east-1 region because CloudFront is a global service. WAF for ALB is created in the same region as the ALB.

## 12. WAF: How to Configure

### WAF has three building blocks:

- **Web ACL** — Container that holds all rules. Attached to CloudFront or ALB. One per resource.
- **Rules** — Individual checks (rate limit, IP block). Each has an action: Allow, Block, or Count.
- **Rule Groups** — Collection of related rules. AWS provides free pre-built managed groups.

### Recommended AWS Managed Rule Groups (Free)

| Rule Group                | Protects Against                             | WCU |
|---------------------------|----------------------------------------------|-----|
| Amazon IP Reputation List | Known malicious IPs                          | 25  |
| Core Rule Set (CRS)       | SQL injection, XSS, path traversal, bad bots | 700 |
| Known Bad Inputs          | Log4j exploits, known attack patterns        | 200 |
| SQL Database              | SQL injection specifically                   | 200 |

### Custom Rate Limiting Rules

Rate-based rules throttle traffic per IP. You define a threshold (e.g., 2000 requests per 5 minutes). If one IP exceeds it, WAF blocks that IP until the rate drops.

| Rule Name      | Limit    | Window | Scope           | Purpose             |
|----------------|----------|--------|-----------------|---------------------|
| RateLimitPerIP | 2000 req | 5 min  | All traffic     | General throttle    |
| LoginRateLimit | 20 req   | 5 min  | /api/auth/login | Prevent brute force |
| APIRateLimit   | 500 req  | 5 min  | /api/*          | API-specific limit  |

### Rule Priority (Evaluation Order)

Rules are evaluated in priority order (lowest number first). First matching rule wins. If no rule matches, the default action applies (usually Allow).

| Priority | Rule                          | Action |
|----------|-------------------------------|--------|
| 0        | Rate Limit per IP (2000/5min) | Block  |
| 1        | Login Rate Limit (20/5min)    | Block  |
| 2        | Geo-Block (optional)          | Block  |
| 3        | AWS IP Reputation List        | Block  |
| 4        | AWS Core Rule Set             | Block  |
| 5        | AWS Known Bad Inputs          | Block  |
| 6        | AWS SQL Injection             | Block  |

| Priority | Rule            | Action |
|----------|-----------------|--------|
| Default  | No rule matched | Allow  |

## Handling False Positives

AWS managed rules may block legitimate requests (e.g., a form submission containing SQL-like text). Always deploy new rules in COUNT mode first — they log without blocking. Monitor logs for 1–2 weeks to identify false positives, add exceptions, then switch to BLOCK mode.

## 13. ALB Security: Restricting to CloudFront

Since ALB is public-facing, anyone who discovers its URL can call it directly, bypassing CloudFront and WAF. You must restrict ALB to accept only traffic from CloudFront.

### Method 1: Custom Header Verification

CloudFront injects a secret header into every request. ALB checks for this header and rejects requests without it. Combined with HTTPS between CloudFront and ALB, the header is encrypted in transit and cannot be sniffed.

```
CloudFront Origin Custom Headers:  
  X-CloudFront-Secret: my-random-secret-value-xyz123  
  
ALB Listener Rules:  
  Rule 1: IF header X-CloudFront-Secret = correct value  
          THEN forward to Target Group  
  Default: Return 403 Forbidden
```

### Method 2: Security Group Prefix List

Restrict the ALB security group to only accept traffic from CloudFront IP addresses using the AWS managed prefix list. This blocks traffic at the network level before it reaches ALB.

```
ALB Security Group Inbound Rule:  
  Port: 443  
  Source: com.amazonaws.global.cloudfront.origin-facing  
  (AWS managed prefix list - auto-updates)
```

**Best practice:** Use BOTH methods together. Security group blocks non-CloudFront IPs at the network level. ALB listener rule verifies the secret header. HTTPS encrypts the header. Three layers of defense.

# 14. CloudFront VPC Origins (Best Option)

CloudFront VPC Origins (launched 2024) eliminates the need for a public ALB entirely. CloudFront connects directly to an internal ALB inside your VPC through a private path. No public exposure, no header tricks, no prefix lists needed.

Traditional (public ALB):  
CloudFront ==HTTPS==> ALB (public) --> ECS  
                                  ^  
                          on the internet  
                  hackers can find it

VPC Origins (private ALB):  
CloudFront ==VPC Origin==> ALB (internal) --> ECS  
                                  ^  
                  NOT on internet  
                  cannot be reached directly

| Setup                        | ALB Type        | Security | Needs Header? |
|------------------------------|-----------------|----------|---------------|
| Public ALB (no restriction)  | Internet-facing | Poor ■   | N/A           |
| Public ALB + header + prefix | Internet-facing | Good ■   | Yes           |
| VPC Origins + internal ALB   | Internal        | Best ■   | No            |

**Recommendation:** Use VPC Origins if available. It provides the strongest security with the simplest configuration. The ALB is completely private and only reachable through CloudFront.

## 15. Complete Security Stack

Even without API Gateway, this architecture provides production-grade security through multiple layers working together.

| Layer | Component         | Protection                                                    |
|-------|-------------------|---------------------------------------------------------------|
| 1     | CloudFront        | TLS termination, DDoS (AWS Shield), geo-restriction, caching  |
| 2     | WAF on CloudFront | Rate limiting, SQL injection, XSS, bot detection, IP blocking |
| 3     | ALB               | Load balancing, health checks, header verification, TLS       |
| 4     | Security Groups   | ALB: CloudFront IPs only. ECS: ALB only. RDS: ECS only        |
| 5     | Application (ECS) | JWT auth, per-user rate limiting, input validation, CORS      |

### What This Pattern Lacks vs API Gateway

| Feature                   | CloudFront + WAF   | API Gateway |
|---------------------------|--------------------|-------------|
| Per-IP rate limiting      | ■ WAF              | ■ Built-in  |
| Per-API-key rate limiting | ■ Need app code    | ■ Built-in  |
| API key management        | ■ Need app code    | ■ Built-in  |
| Usage plans (tiers)       | ■ Not available    | ■ Built-in  |
| Request transformation    | ■ Not available    | ■ Built-in  |
| Developer portal          | ■ Build separately | ■ Built-in  |

**Conclusion:** CloudFront + WAF + ALB is sufficient for web applications where users access via browser. API Gateway is needed only when you sell a public API and need per-key rate limiting, usage plans, and a developer portal.



## 16. Complete Request Flow

### Frontend Request: [myapp.com/dashboard](http://myapp.com/dashboard)

- Route 53 resolves myapp.com → CloudFront
- CloudFront matches default behavior (\*) → S3 origin
- CloudFront checks cache → HIT: return cached file instantly
- MISS: fetch from S3 via OAC → cache at edge → return to user
- If /dashboard not in S3 → 403 → custom error response → serve index.html
- React Router handles /dashboard on client side

### API Request: [myapp.com/api/orders](http://myapp.com/api/orders)

- Route 53 resolves myapp.com → CloudFront
- WAF checks: rate limit ■, not blocked ■, no SQL injection ■
- CloudFront matches /api/\* behavior → ALB origin
- Cache: DISABLED → every request forwarded to ALB
- CloudFront adds X-CloudFront-Secret header
- HTTPS to ALB → ALB verifies header → routes to healthy ECS task
- ECS processes request → response flows back: ECS → ALB → CloudFront → user

## 17. Interview-Ready Answers

### Q: How do you serve frontend and API from one CloudFront?

"I configure two origins: S3 for the React frontend and ALB for the backend API. Two behaviors route by path — /api/\* goes to ALB with caching disabled, and the default path goes to S3 with caching enabled. For SPA routing, custom error responses serve index.html for 403/404 so React Router handles client-side routes. One domain eliminates CORS issues."

### Q: Where does TLS termination happen?

"TLS terminates at CloudFront for user-facing traffic. I also enable HTTPS between CloudFront and ALB because CloudFront injects a secret header for verification — without HTTPS, this header could be sniffed. ALB terminates TLS using an ACM certificate and forwards plain HTTP to ECS tasks inside the private VPC."

### Q: How does WAF work with CloudFront?

"WAF attaches to CloudFront and blocks traffic at the edge before it reaches the backend. I use free AWS managed rule groups for SQL injection, XSS, and IP reputation. I add custom rate-based rules for per-IP throttling and endpoint-specific limits like 20 requests per 5 minutes on the login endpoint. New rules deploy in COUNT mode first to catch false positives."

### Q: How do you secure the ALB behind CloudFront?

"Three layers: Security group restricts ALB to CloudFront IPs using the AWS managed prefix list. HTTPS between CloudFront and ALB encrypts the custom header in transit. ALB listener rule verifies the header and returns 403 for requests without it. For the best security, I use CloudFront VPC Origins with an internal ALB, eliminating public exposure entirely."

### Q: When do you need API Gateway in addition to CloudFront?

"CloudFront + WAF + ALB is sufficient for web applications where users access via browser. API Gateway is needed when you have a public API consumed by external developers and require per-API-key rate limiting, usage plans, request transformation, or a developer portal. For most web apps, CloudFront with WAF provides equivalent security at lower cost and complexity."